Week 10 Extra: Perceptron Training and XOR

Distinction-tier supplement to the gradient-descent lecture

FIT1061: Introduction to Artificial Intelligence

2026-07-04

Table of contents

1 Why this supplement exists 1

2 Where we left off 1

3 The perceptron learning rule 2
- 3.1 The same downhill step, one example at a time . . . 2
- 3.2 The training loop, structurally . . . 2

4 A worked example: one epoch on XOR 3

5 Linear separability, and why XOR breaks it 4

6 What a fix looks like 5

7 Aiming for a Distinction: what to do 5

8 Going further 5

9 Optional viewing 6

10 References 6

1 Why this supplement exists

The Week 10 lecture covers **gradient descent** in full: the derivative, the update rule, the learning rate, and the move into two dimensions. That's enough to complete the Pass task this week.

These supplementary notes cover the algorithm the lecture only points at in its closing slide: the **perceptron learning rule** and what happens when you turn it loose on **XOR**. If you're aiming for a Distinction, this is required reading. The Distinction task asks you to implement the perceptron update, train it on a linearly separable dataset, watch it converge, then run it on XOR and discover, numerically and geometrically, that a single straight line cannot solve the problem.

If you've already met the perceptron through other reading, you can skim. The point of these notes is to pin the ideas down precisely, connect the perceptron back to the gradient descent you just learned, and make the XOR failure something you work out in numbers yourself.

2 Where we left off

Two weeks of the machine-learning block set this up.

- In the **weighted-sums** week you computed a prediction $\hat{y} = \text{sign}(\mathbf{w} \cdot \mathbf{x} + b)$ from **fixed** weights **w** and bias b. You drew the decision boundary $\mathbf{w} \cdot \mathbf{x} + b = 0$ and read off which side each point landed on. But the weights were handed to you.
- This week's lecture answered a different question (*how does a machine improve a setting once it can measure how wrong it is?*) with the update rule $x \leftarrow x - \alpha\, f'(x)$: step opposite the slope, scaled by the learning rate.

The obvious question left open: **where do the weights in a classifier come from?** The perceptron is the original answer (Rosenblatt, 1958) and, as we'll see, it is the *same downhill idea* the lecture just taught, applied one training example at a time.

3 The perceptron learning rule

A perceptron predicts with the weighted-sum rule you already know:

$$\hat{y} = \text{sign}(\mathbf{w} \cdot \mathbf{x} + b), \qquad \text{sign}(z) = \begin{cases} +1 & z \geq 0 \\ -1 & z < 0. \end{cases}$$

Training presents one labelled example $(\mathbf{x}, y)$ at a time and nudges the weights **only when the prediction is wrong**:

$$\mathbf{w} \;\leftarrow\; \mathbf{w} + \eta\,(y - \hat{y})\,\mathbf{x}, \qquad b \;\leftarrow\; b + \eta\,(y - \hat{y}).$$

Here η (eta) is the learning rate: the same role α played in the lecture. Because both y and $\hat{y}$ live in $\{-1, +1\}$, the error term $(y - \hat{y})$ is only ever 0 (correct: no change), $+2$ (predicted -1, should have been $+1$), or -2 (the reverse). So each mistake pushes the boundary a fixed amount **toward** getting that point right.

3.1 The same downhill step, one example at a time

Rewrite the weight update as

$$\mathbf{w} \;\leftarrow\; \mathbf{w} - \eta\, \underbrace{(\hat{y} - y)\,\mathbf{x}}_{\text{error} \times \text{input}},$$

using $(\hat{y} - y) = -(y - \hat{y})$: the plus-sign rule above and this minus-sign form are the *same* update, just written to look like a descent step.

Compare with the lecture's $x \leftarrow x - \alpha\, f'(x)$. The shape is identical: **a parameter, minus a learning rate times a quantity that measures the error.** So the perceptron is gradient descent's ancestor. One honest caveat, because the question always comes up: it is *not* descending the raw count of misclassifications; that 0/1 error is flat, with zero slope almost everywhere, so there is nothing to roll down. It descends a *proxy* for it (the "perceptron criterion" $\max(0,\, -y(\mathbf{w} \cdot \mathbf{x} + b))$, a measure of how far a mistake is on the wrong side), which is exactly why the update fires **only on a mistake**. The same online, one-example-at-a-time idea (later done in small batches) is what scales up, decades on, to training AlexNet.

3.2 The training loop, structurally

You implement this loop yourself in the task, so here is its shape rather than code to lift. Initialise the weights **w** and the bias b at zero, then repeat for a fixed number of **epochs** — one epoch is a single sweep through all the training points, shuffled each sweep so the order of the data doesn't bias the path:

- Predict each point with the sign rule, $\hat{y} = \text{sign}(\mathbf{w} \cdot \mathbf{x} + b)$.
- **Only when that prediction is wrong**, apply the update rule from above — $\mathbf{w} \leftarrow \mathbf{w} + \eta\,(y - \hat{y})\,\mathbf{x}$, with the matching bias step. A correct prediction changes nothing.

- After each full sweep, record the accuracy. That sequence of numbers is the **learning curve**, and it is what tells XOR's story.

Three moving parts, then: a prediction, an update that fires only on a mistake, and one accuracy reading per sweep. The worked example below traces a single epoch by hand so you can check your own numbers against it.

4 A worked example: one epoch on XOR

XOR labels the four corners of the unit square. The two corners on one diagonal are class $+1$; the two on the other diagonal are class -1:

point **x**	label y
$(0, 0)$	-1
$(0, 1)$	$+1$
$(1, 0)$	$+1$
$(1, 1)$	-1

Start from **w** $= (0, 0)$, $b = 0$, learning rate $\eta = 1$, and present the points in the order above (no shuffle, so you can check every number by hand). Recall sign$(0) = +1$.

present **x**	**w** · **x** $+ b$	$\hat{y}$	y	$y - \hat{y}$	new **w**	new b
$(0, 0)$	0	$+1$	-1	-2	$(0, 0)$	-2
$(0, 1)$	-2	-1	$+1$	$+2$	$(0, 2)$	0
$(1, 0)$	0	$+1$	$+1$	0	$(0, 2)$	0
$(1, 1)$	2	$+1$	-1	-2	$(-2, 0)$	-2

After this single epoch **w** $= (-2, 0)$, $b = -2$. Check the four points against this boundary and you get $(0, 0) \to -1$ (correct), $(0, 1) \to -1$ (wrong), $(1, 0) \to -1$ (wrong), $(1, 1) \to -1$ (correct): **2 out of 4**. Run another epoch and the numbers move again, but the accuracy never climbs to $4/4$: the boundary just keeps jumping around the square. That non-convergence *is the result*: on XOR the learning curve rattles around and never reaches 1.0, no matter how many epochs you allow.

Here is *why* it rattles rather than settling. Each mistake shoves the fence just far enough to rescue the corner it currently gets wrong, rotating and shifting the line to put that one corner on the right side. But the two classes interleave on opposite diagonals, so pulling the fence over to save a $+1$ corner strands one of the -1 corners on the wrong side (and vice versa). The next mistake shoves the fence back to rescue *that* corner, which re-strands the first. There is no resting position where every corner is correct at once, so the line never stops moving: it cycles forever through a small set of near-miss configurations, and the accuracy trace saw-tooths up and down instead of climbing to 1.0.

This hand-trace fixes the presentation order so you can check every number, but your task run shuffles the points each epoch, so its accuracy wanders differently from run to run. The one guaranteed fact is that no epoch ever reaches $4/4$. The *peak* is shuffle-dependent: many runs top out at $3/4$ (75%), but a sizeable fraction (well over one run in five, across seeds) never do better than $2/4$ (50%), so a 50% ceiling is not a bug either. Either way the count of epochs at $4/4$ stays 0.

Contrast this with the separable Gaussian clusters in the task's `SEPARABLE_POINTS`: there the same loop reaches 100% accuracy within roughly one epoch (your exact final weights depend on your seed and the shuffle order): once it stops making mistakes it stops updating, so the boundary is frozen from that point on. Same code, same rule; the difference is entirely in the **shape of the data**.

5 Linear separability, and why XOR breaks it

Definition: linear separability

A labelled dataset in the plane is **linearly separable** if there exists a single straight line (in general, a hyperplane $\mathbf{w} \cdot \mathbf{x} + b = 0$) with every $+1$ point on one side and every -1 point on the other.

Picture the $+1$ and -1 points as red and blue sheep standing in a field. A perceptron is allowed exactly **one** straight electric fence. "Linearly separable" just means that one straight fence can pen every red sheep on one side and every blue sheep on the other, with none of the wrong colour trapped alongside them. For XOR the two colours stand on opposite diagonals of the square, so no single straight fence can do it: wherever you run the fence, one sheep ends up on the wrong side. Relabel the very same four corners to AND or OR, though, and one fence suddenly suffices, because now a single colour sits off in its own region.

A perceptron *can only draw one straight line.* The prediction $\text{sign}(\mathbf{w} \cdot \mathbf{x} + b)$ splits the plane into exactly two half-planes. So a perceptron can learn a dataset **if and only if** that dataset is linearly separable.

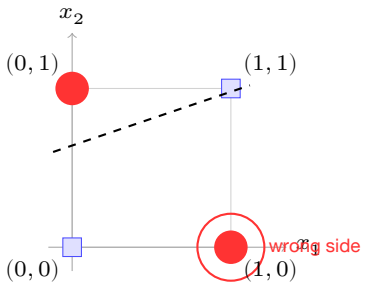
The sharpest way to see this is to hold XOR's four corners fixed and just relabel them. Label $(1, 1)$ as $+1$ and the other three as -1: that is **AND**, and a single straight line separates the lone $+1$ corner from the rest. Label $(0, 0)$ as -1 and the other three as $+1$: that is **OR**, again one line does it. Feed either labelling to the *identical* perceptron loop and it converges cleanly: the AND relabelling, for instance, reaches 100% in a handful of epochs. The points and the algorithm are unchanged; only the labelling moved. So when XOR fails, the culprit is the **labelling geometry**. The data volume is fine and so is the code; the labels are simply arranged so that no straight line can win. (The separable Gaussians make the same point with different data, which is what you'll actually run.)

XOR is not separable. Its $+1$ points sit at $(0, 1)$ and $(1, 0)$: opposite corners of the square. Its -1 points sit at $(0, 0)$ and $(1, 1)$: the *other* diagonal. Any straight line that puts both $+1$ corners on one side is forced to put at least one -1 corner there too: the two classes interleave around the square, so no single cut separates them. There is no $(\mathbf{w}, b)$ that works, so the training loop can never stop making mistakes.

Why the bias does not rescue it

A tempting thought: "the line can't separate them, but surely the bias b gives an extra degree of freedom that fixes it." It doesn't. The bias only **shifts** the line; the weights only **rotate** it. Together they let you place *any* straight line anywhere, and we've just argued that *no* straight line separates XOR. Adding b enlarges the family of lines, but XOR's failure is that the whole family of straight lines is the wrong tool. You need a boundary that isn't a single straight line.

Here is the whole argument in one picture: the four corners, coloured by label, and one candidate line that (like every other) strands a corner on the wrong side:



Red circle = class $+1$ (XOR true); blue square = class -1. The classes sit on *opposite diagonals*, so wherever you place the line, a corner is stranded on the wrong side. Relabel to AND or OR and a single line *does* separate them: same points, only the labelling changed.

6 What a fix looks like

XOR becomes learnable the moment you leave the family of single straight lines. Three routes, all of which you'll meet later in your degree:

- **A hidden layer.** Stack two perceptrons feeding into a third. Concretely, XOR $=$ OR AND NAND: the first layer can learn an OR line and a NAND line (NAND = "not both"), and the second layer ANDs the two together. That combination is exactly XOR: two learned lines carving the diagonal pattern that one line cannot. This is the multilayer network, and training it is what gradient descent (via backpropagation) unlocked. One catch: the perceptron's own sign step has zero gradient almost everywhere and an undefined jump at 0, so there is no usable slope to follow through a hidden layer. That is why multilayer training replaces sign with a **differentiable** activation such as the sigmoid (a smooth S-shaped curve) which is precisely what lets gradient descent (backpropagation) reach the hidden weights.
- **A non-linear feature.** Add a third coordinate to each point, for example the product $x_1 x_2$. In that richer space the four XOR corners *become* linearly separable, and a plain perceptron handles them. You solved a curved problem by lifting it into more dimensions where it's flat again.
- **A kernel.** The same lift, done implicitly, so you never have to write the extra feature out by hand. This is the idea behind support-vector machines.

The throughline: XOR is hard because of the *model*, not the *data*. There are only four points and none are noisy; a single straight line just cannot carve the diagonal pattern. The 1969 critique of the perceptron (Minsky and Papert's *Perceptrons*) turned exactly on this example, and the field's answer (more layers, trained by gradient descent) is the lecture's whole arc from one tuned line to millions.

7 Aiming for a Distinction: what to do

The Distinction task ships a helper module (`d3_helpers.py`) with the datasets (`SEPARABLE_POINTS`, `XOR_POINTS`) and the plotting functions (`plot_boundary_evolution`, `plot_learning_curve`). Add your training code to the starter notebook, then:

1. **Implement `predict_all` and `train_perceptron`** following the pseudocode above. Use the update rule exactly: $\mathbf{w} \leftarrow \mathbf{w} + \eta(y - \hat{y})\mathbf{x}$, with the matching bias update, and only when the prediction is wrong. Snapshot `(epoch, weights, bias)` each epoch so you can plot the boundary evolving.
2. **Train on the separable Gaussians.** Plot the learning curve (it should hit 100% in about one epoch) and the boundary evolution. Because it converges so fast, the interesting snapshots show the jump from the zero-vector start to the converged line; once accuracy hits 100% the perceptron stops updating and later snapshots are identical.
3. **Run it on XOR** for at least 100 epochs. Plot the learning curve (it will never reach 1.0) and the boundary evolution, where you'll watch the line jump around the unit square. Report the min/max accuracy and the count of epochs at 100%. The one guaranteed sanity check is that no epoch ever reaches $4/4$: the count of epochs at 100% must be 0. The peak accuracy is shuffle-dependent: many runs top out at $3/4$ (75%, three of four corners right), but a sizeable fraction top out at only $2/4$ (50%), so a 50% ceiling is not a bug, just a different shuffle.
4. **Write the geometric explanation** (300–500 words): what linear separability is, why XOR specifically violates it, what the bias does and doesn't do, and one concrete fix (hidden layer, non-linear feature, or kernel) with a sentence or two on why it works.

The marking rubric is in the Distinction task notebook. At this tier you may use AI to explain concepts or debug a specific error, but you may **not** use it to generate the update rule, the training loop, the experiments, or the geometric write-up. Cite all AI use.

8 Going further

The perceptron is one of the oldest ideas in the field, and its failure on XOR is one of the most consequential.

- **Rosenblatt (1958)** introduced the perceptron. Its *convergence theorem* – if the data is linearly separable, the learning rule is *guaranteed* to find a separating boundary in finitely many updates (rigorously proved by Novikoff, 1962) – says nothing when the data isn't separable, which is precisely XOR's loophole.
- **Minsky and Papert (1969)**, *Perceptrons*, proved the single-layer perceptron's limits rigorously. The book is often blamed for the first "AI winter" in neural-network research.
- **Rumelhart, Hinton and Williams (1986)** popularised **backpropagation** (gradient descent through a multilayer network) which trains exactly the hidden-layer fix that XOR needs. This is the "the method dates to 1986" line from the lecture.
- **Krizhevsky, Sutskever and Hinton (2012)**, AlexNet, is what happens when that same machinery meets a million images and fast hardware. The lecture's closing line (*the perceptron tuned one line; gradient descent tunes millions*) is this whole history in one sentence.

9 Optional viewing

You do not need to watch these; the notes above stand on their own. They are here only for students who find an animated picture easier to hold onto than prose, since these notes replace a video that was never recorded.

- **Primary (optional, interactive):** TensorFlow Playground on the XOR ("exclusive or") dataset. Pick the XOR dataset, keep only the two raw inputs x_1, x_2 (no extra features), and set zero hidden layers: you get exactly one straight boundary, and you can watch it fail to separate the diagonals no matter how long it trains, the precise picture these notes replace. Then add a hidden layer, or switch on the $x_1 x_2$ feature, and watch it succeed.
- **Backup (optional):** 3Blue1Brown, *But what is a neural network?* (Neural networks, Ch. 1). A hierarchical-feature network on handwritten digits; no perceptron and no XOR, but a clear picture of how weighted sums stack into layers, the building block behind the hidden-layer fix above.

10 References

- Rosenblatt, F. (1958). The perceptron: A probabilistic model for information storage and organization in the brain. *Psychological Review*, 65(6), 386–408.
- Minsky, M., & Papert, S. (1969). *Perceptrons: An Introduction to Computational Geometry.* MIT Press.
- Rumelhart, D. E., Hinton, G. E., & Williams, R. J. (1986). Learning representations by back-propagating errors. *Nature*, 323, 533–536.
- Krizhevsky, A., Sutskever, I., & Hinton, G. E. (2012). ImageNet classification with deep convolutional neural networks. *NeurIPS 25*.
- Russell, S., & Norvig, P. (2020). *Artificial Intelligence: A Modern Approach* (4th ed.), §19.6 (linear classifiers) and §21 (deep learning).